Development Plan
Natural Control

Group 7
Lucas DeBoer
Kevin Kim
Aaron Loh
Curtis Vos

September 28, 2026

Table 1 summarizes the revision history.

Table 1: Revision history.

Date	Contributor	Change
2026-09-23	Aaron Loh / AI assistance	Initial review draft based on Aaron's planning interview; team ratification pending.
2026-09-27	Lucas DeBoer	Reviewed the contents.
2026-10-02	Aaron Loh / AI assistance	Draft revisions for issues 11 and 12; browser/account scope.

This plan describes how a four-person team proposes to build and evaluate a general-purpose, voice-supervised agent interface for blind users. It covers team operations, technical choices, the proof of concept (POC), and evaluation. It does not promise a universal replacement for screen readers. Initial use will be on a Mac, in English, alongside VoiceOver and keyboard access. The companion Problem Statement and Goals defines the user outcomes. The initial supported scope is browser interaction and connected-account tools; native Mac application and general local-file control are deferred.

1 Confidential Information?

No external confidentiality agreements are present for any team members. Participant identities, recordings, account credentials, and personal task data are private even without an industry agreement. Public artifacts will use test accounts, synthetic data, and de-identified findings shared with participants' consent. Raw recruitment and meeting materials remain outside the public repository. No raw microphone audio will be retained by default; provider retention policies require separate review and disclosure.

2 Intellectual Property (IP) to Protect

No external intellectual-property agreement is currently known. The intended contribution is the accessible interaction and supervision layer, its adapters, and evaluation, building on existing models and tools. The team must confirm ownership and any required institutional agreements before release. Upstream code, model weights, services, and assets retain their own terms; licensing our code does not relicense dependencies.

3 Copyright License

MIT is the license for original project contributions. Its permissive terms require retaining the copyright and permission notice [**?**]. The repository's LICENSE contains the team copyright and preserved upstream template notice. Third-party notices will be recorded when actual dependencies are selected, rather than assuming all candidate tools share MIT.

4 Team Meeting Plan

The plan is one weekly team sync, supported by asynchronous coordination on the team's Discord server as required. Weekly team syncs will be held in person every Wednesday from 2:30 to 3:30 p.m. Toronto time. The meeting location will be confirmed virtually. Each meeting has a rotating chair and note taker, selected at the previous meeting. The meeting chair will create an issue-based agenda at least 24 hours prior to the meeting. Members review materials beforehand. Attendance, decisions, owners, and next commitments are recorded on GitHub. A member unable to attend can provide an advance written update and input on decisions; emergencies are communicated when practical.

5 Team Communication Plan

Discord is proposed for coordination, with a response to actionable messages within one working day. Decisions, acceptance criteria, blockers, and assignments move to GitHub issues so they remain discoverable. Sensitive participant details and secrets do not belong in either public issues or the project board. Urgent blockers are explicitly flagged; silence is not agreement or approval.

6 Team Member Roles

The team has four members: Aaron Loh, Curtis Vos, Kevin Kim, and Lucas DeBoer. Each member will take ownership of a primary workstream and be a cross-reviewer for another workstream. Primary ownership provides accountability without preventing another member from contributing. All four members implement, test, review, and document work. Owners and cross-review assignments are revisited at milestones.

Role changes may occur between milestones. If a member is unable to fulfill their responsibilities or wishes to change roles, they must follow these guidelines:

- the owner tells the team in Discord as soon as they realize they need a change - explaining which specific tasks are affected.
- The team chooses whether to split the work or assign a new owner based on availability.
- The cross reviewer doesn't automatically inherit both roles.
- The outgoing owner shares progress / blockers with the new owner, and GitHub assignments and deadlines are updated.
- As a group we check at the next weekly meeting whether the new arrangement is working.

Table 2 summarizes proposed workstreams.

Table 2: Proposed workstreams.

Proposed workstream	Responsibility and assignment
Voice and accessible client	Conversation controls, keyboard access, VoiceOver compatibility. Owner: Lucas DeBoer. Cross-reviewer: Curtis Vos.
Agent and tool integration	Hermes adapter, account/browser tools, execution control. Owner: Aaron Loh. Cross-reviewer: Kevin Kim.
Deployment and reliability	Configuration, credentials, CI, reconnection, observability. Owner: Kevin Kim. Cross-reviewer: Aaron Loh.
Evaluation and quality	Recruitment coordination, task protocol, outcome measurement, accessibility checks. Owner: Curtis Vos. Cross-reviewer: Lucas DeBoer.

Meeting chair, note taker, release coordinator, and external liaison (if required) are rotating process roles. Curtis Vos is the liaison between course staff and the team. A contributor reviews outside their primary workstream to reduce dependence on one person.

7 Workflow Plan

Work is tracked in the public repository. Each meaningful change starts with an issue describing the problem, acceptance criteria, owner, and relevant risk. The existing issue templates (Lectures, TA Meetings, etc.) should be used when possible. Labels distinguish features, defects, accessibility, research, documentation, and blockers. Short-lived topic branches and small pull requests keep changes reviewable. Branch names and pull request titles should include an issue number if relevant. One non-author reviewer must approve each pull request; relevant continuous integration (CI) checks must pass before merging into the protected main branch. Tags will be used to mark major milestones as they are completed.

The repository already contains a LaTeX/PDF and Pages workflow. Python and TypeScript checks will be added when the corresponding implementation exists: format/lint, type checks, unit/contract tests, and applicable integration tests. Passing automated checks does not establish accessibility. Manual VoiceOver/keyboard checks and participant evidence remain necessary. Secrets stay out of source control and logs. A change is done when its acceptance criteria, checks, documentation, and review are complete, with known limitations recorded.

For each deliverable, the quality-assurance owner coordinates reviews against the course checklist and Avenue rubric. Issues found during review are recorded and addressed before submission.

8 Project Decomposition and Scheduling

A GitHub project will be used to track development and milestone progress. The project is a Kanban board with Backlog, Ready, In progress, Review, and Done columns. Each member commits to concrete weekly outputs rather than fixed hours or commit quotas. Dependencies and risks are recorded on issues. The team aims for one primary active item per member and resolves blocked work at the weekly sync.

Drafts are ready 48 hours before deadlines. For the first milestone, the target is September 26 at 11:59 p.m. Toronto time, leaving time for human review and ratification before September 28 at 11:59 p.m. The TA's September 23 guidance was that assessment uses the repository's latest push at the deadline, without an Avenue upload; the knowledge base links the meeting source. Later official announcements take precedence.

Large tasks will be divided into GitHub issues with a clear description, an assigned owner, and acceptance criteria that define when the work will be complete. Each issue should generally require hours of work instead of days. Assignments will consider members' availability, relevant experience, and dependencies on other work.

Table 3 provides an overview of the milestones.

Table 3: Milestone overview.

Period/date	Planned output or course milestone
September 28, 2026	Problem Statement and Goals; Development Plan including POC and charter.
October 1	First milestone peer-review issues; substantive findings, not manufactured issue counts.
October 19	Software Requirements Specification (SRS) and Hazard Analysis; discovery informs scope and safety requirements.
October–November	Working voice/agent supervision loop; instrumented POC trials; repeated discovery if recruitment succeeds.
November 2	Verification and Validation (V&V) Plan.
November 16	Design revision minus one and proposed fall Interview Report, as reported approved in the team review.
November 23–December 3	Course POC demonstration window; assigned slot pending.
January 18, 2027	Design documentation; integration and accessibility refinement continue.
March 8	V&V report and proposed winter Usability Report, as reported approved in the team review.
March 31	Final documentation.
April 6	EXPO; event details to confirm.

Dates reflect the checked course calendar snapshot [**?**]; this is a milestone overview, not a replacement for the complete course calendar.

Resources and budget. The proposed total budget has a Canadian-dollar ceiling of $500. A provisional allocation is $240 participant appreciation, $120 model/voice application programming interfaces (APIs), $60 hosting/telephony, and $80 reserve. Appreciation assumes at most twelve paid sessions at $20, subject to recruitment and institutional guidance. Team Macs are already available. Test accounts and synthetic data reduce exposure of personal information. Usage metering and monthly reconciliation will track spend; paid expansion requires a revised allocation within the ceiling or an explicitly approved budget change.

9 Proof of Concept Demonstration Plan

The POC tests whether the voice interface can supervise a running agent. It is a technical demonstration, separate from the later participant evaluation.

9.1 Risk and experiment

The central risk is whether a visually impaired user can understand and control a running general-purpose agent through voice without visually monitoring it. Individual speech and agent components do not establish that their integration works. The POC will test the supervision interface across two task classes, rather than implement a set of website-specific scripts:

1. **Email/calendar tools:** use synthetic correspondence to prepare a reply or calendar entry; clarify ambiguous intent and obtain approval before sending or creating the event.
2. **An unfamiliar website:** locate and compare information through general browser tools, report a meaningful choice, and incorporate a correction. The site and task are selected before the trial, without coding a dedicated workflow.

The demonstration uses one short task from each class. Across these tasks, exercise clarification, consequential-action approval, progress, correction, explicit execution stop, and reconnection at least once. A full repetition of every condition in every class belongs to later evaluation. The demonstration uses controlled accounts and data; no participant's real email is required.

9.2 Control semantics

Ordinary speech interruption stops spoken output while the underlying task continues. An explicit "stop working" requests cancellation of execution. The interface first acknowledges receipt, then separately confirms whether execution has stopped. Acknowledgment is not proof of cancellation. Once execution is confirmed to have stopped, no further action may be dispatched for that task. An action already completed cannot be undone by saying stop; in-flight actions with an unknown outcome must be reported and reconciled before retrying.

Disconnection pauses new actions unless the user previously authorized background continuation for that task. Reconnection presents current state and pending decisions; it must not duplicate a send or other consequential action. Navigation and preparation can proceed autonomously, but sending, purchasing, submitting, and consequential account/file changes require specific approval. If the action or its relevant details change, the previous approval does not apply.

The POC will inject a misunderstanding, interrupt speech, request a stop during a tool operation, and disconnect while work is active. Logs correlate task, action, approval, cancellation, and connection events without retaining sensitive payloads. The team must verify cancellation through the adapter and downstream worker; account and browser actions may have different cancellation behavior.

9.3 Later evaluation and provisional targets

The following participant study and performance targets guide subsequent validation; they are not prerequisites for the technical POC. Targets below are provisional goals, not results:

- At least 80% correct independent completion on a declared representative task set, following assisted setup. Correct means the specified outcome was achieved and verified without researcher task assistance. Report numerator, denominator, task categories, failures, and any assistance separately.
- Stop acknowledgment within two seconds, measured from the end of a recognized explicit stop utterance to audible acknowledgment. Record recognition failures separately and measure execution-stop confirmation latency independently; no two-second execution-cancellation guarantee is claimed.
- Approval before every tested consequential action, checked against action and approval traces, including attempts that should be blocked.
- Assess whether users can identify completion or a blocker and correct a seeded misunderstanding. Record errors, clarification turns, time, perceived control, and useful versus excessive spoken feedback.

The team aims to recruit two to four blind participants with screen-reader experience for repeated discovery and evaluation. Recruitment is unconfirmed. The protocol, consent, and institutional research requirements must be resolved before sessions. A personal contact may facilitate an introduction to an accessibility organization; this is not a partnership or recruitment commitment.

Before the main evaluation, freeze task definitions, outcome checks, allowed assistance, and exclusions. Report all attempted tasks and explain invalidated trials. Compare participants' usual screen-reader workflow with the assistant where feasible, varying order to reduce learning effects. A small convenience sample supports formative findings, not population-wide efficacy claims. Technical POC results and participant results will be reported separately.

9.4 Failure consequences

Table 4 summarizes POC uncertainties and failure consequences.

Table 4: POC uncertainties and failure consequences.

Uncertainty	Evidence needed and consequence if unresolved
Harness supervision	Verify streamed status, approvals, steering, cancellation, and reconnection on a pinned Hermes version. If unavailable, revisit the adapter/harness before promising supervision.
Tool cancellation	Test slow and in-flight account/browser operations. If the outcome is unknown, expose that state and block retries; do not claim confirmed stop.
Browser context	Test structured page information and screenshots on unfamiliar sites. Missing labels or ambiguous visuals may prevent reliable action; report the limitation and request clarification rather than assume success.
Voice and VoiceOver	Measure latency, intelligibility, and competing audio with keyboard alternatives. If remote calls fail, retain local voice.
Remote continuity	Test network loss, pending approvals, and duplicate prevention. Defer telephone interaction if it cannot preserve control.
Recruitment	Confirm access early. Without participants, report technical feasibility only and disclose the missing usability evidence.

10 Expected Technology

The proposed architecture is:

> Accessible TypeScript/React web client → LiveKit/Python integration → Hermes → account tools and Browser Use.

The team will implement the accessible task/decision presentation and supervision adapter, reusing existing speech, agent, and tool implementations. One supported default configuration will be documented and version-pinned, with extension points and user-provided credentials. Provider choices and exact versions remain pending POC evidence. No model training is planned.

10.1 Agent and action tools

Hermes is the primary harness candidate. Its documented programmatic interface includes streaming events, steering, interrupt requests, and server requests for clarification/approval [**?**]. These are promising integration primitives, not proof that our end-to-end control semantics already work. The POC must verify actual behavior and failure handling on a pinned release. Codex app server is an alternative with a custom-client integration surface [**?**]; OpenClicky is an implementation reference. Neither establishes that arbitrary desktop plugins or account integrations transfer to our client. OpenClaw is excluded from the proposed stack.

Browser Use is the intended general browser component through Hermes. Hermes documents a Browser Use path and fallback browser mechanisms [**?**]. Packaged browser sessions must not be assumed to share a user's logged-in Chrome profile. Authentication, signed-in browser attachment, permission boundaries, and compatibility need separate trials. Direct account APIs can offer clearer structured outcomes than visual clicking; browser tools remain necessary where suitable APIs are

unavailable. Account connectors must be selected and verified individually, including scopes and credential handling.

Environment inputs and browser perception. Account tools provide structured service data and action results. For websites, the proposed Browser Use integration will obtain structured page information (such as text, controls, and document object model (DOM) state) and screenshots where supported by the selected integration. Structured information helps identify controls and values; screenshots can provide visual context when structure is insufficient. Neither source guarantees correct interpretation of an arbitrary website.

The Browser Use library is open source under the MIT license and can run with a local browser; model inference and optional hosted services have separate costs [**?**]. The POC must verify which context the Hermes adapter actually exposes, how it reports action results, and whether it permits approval before consequential actions and cancellation during delegated browser work. An instruction to ask for approval is not evidence of an enforced action boundary. Relevant progress, choices, and uncertainty will be spoken to the user; receiving page context does not require reading every page element aloud.

Deferred native control. Direct control of native Mac applications, general local-file tasks, and macOS accessibility API integration are outside the initial supported scope and POC. They may be investigated later, with separate permission, perception, and action-verification tests. Using a local browser does not imply permission to control the rest of the computer.

10.2 Voice, client, and deployment

LiveKit with Python is proposed for audio sessions and tool integration [**? ?**]. Python aligns with the anticipated agent-side integration. A TypeScript-only voice backend is an alternative that reduces language count; the Python choice adds a typed cross-process boundary to test. A voice tool call does not itself guarantee cancellation of downstream work.

TypeScript/React is proposed for the web client, using semantic controls, visible text/history, focus management, and full keyboard operation. Familiar web tooling and iteration speed favor it over an initial Swift-only client. React does not guarantee accessibility; VoiceOver testing is required. The web interface and local service support assisted setup followed by independent daily use.

ElevenLabs remains a speech-output candidate through LiveKit's documented plugin [**?**]. Alternatives include other supported speech providers and native system speech. Selection will compare intelligibility, interruption behavior, latency, privacy, and actual cost. Recognition and general reasoning/vision models will likewise use existing providers, with versions and usage recorded. Local models are an option to investigate, not a commitment to fully local performance or zero cost.

Jev is optional for structured evaluation or routing [**?**]. Its documented role does not establish suitability as the primary conversational or visual-control model. A deterministic policy is the simpler alternative for mandatory approval rules. Promotional pricing is not a basis for the project budget.

Deployment remains hybrid: a local Mac, hosted services, or their combination. A locally controlled browser requires an online local component; a hosted browser is an alternative. Hosting the voice/agent services does not remotely confer access to the user's Mac or existing signed-in browser sessions. The default topology, secret storage, provider retention, and network requirements will be documented after a working configuration is demonstrated. A second voice channel connecting telephone and on-Mac interaction is the leading stretch goal.

10.3 Engineering and measurement tools

Git, GitHub, and GitHub Projects provide version history, issues, reviews, and project tracking. GitHub Actions supplies repeatable checks; hosted CI cannot substitute for Mac permission tests or participant evaluation. Proposed Python tools are Ruff, pytest, and pytest-cov; proposed client tools are TypeScript strict checks, ESLint, Prettier, Vitest, and Testing Library. Final configurations and versions will be pinned during setup. Coverage is diagnostic, not proof of correctness.

Contract tests will exercise approval, cancellation, reconnection, and task-state boundaries using controlled workers, followed by real-tool integration trials. Deterministic tests do not imply deterministic website workflows in the product. Monotonic timestamps will measure voice response and stop latency; task traces will measure success, errors, and cost per completed task. A simple instrumented trace is preferable initially to a complex observability service. The team must learn the harness protocol, asynchronous audio integration, and accessible focus management through small tested integration slices.

11 Use of AI and LLMs

The product uses existing models to interpret goals and operate tools. The team also proposes AI assistance for research, code, tests, and documentation, with human understanding and review. All AI-generated content must be reviewed by a team member other than the contributor who prepared it. This review must be explicitly requested by that contributor. Generated code must pass relevant checks and review. Sources must be opened and verified, claims distinguished from assumptions, and participant quotations/results never generated as evidence. Per-document disclosures record material assistance, affected sections, tool versions where known, and actual verification [**?**].

12 Coding Standard

Python will follow PEP 8 conventions, type annotations at public boundaries, and a shared formatter/linter configuration. TypeScript will use strict types and shared formatting. Small modules separate voice, task control, and tool adapters; state transitions and error outcomes are explicit. Tests cover behavior and failure boundaries. Semantic HTML, accessible names, keyboard operation, and focus behavior are part of review. Credentials and participant data are excluded from code, fixtures, and ordinary logs. Exceptions to conventions are documented in the relevant pull request.

Appendix A: Generative AI Use Disclosure

Tool and use. Aaron Loh used OpenAI Codex with GPT-6 Astra, primarily at Medium reasoning and occasionally High (settings reported by Aaron). Codex helped extract questions from course requirements, challenge assumptions, research options, and develop ideas through discussion. It also assisted with drafting and revising text throughout this document, organizing references, and LaTeX/PDF formatting. Aaron made the judgment calls and reviewed the work. During integration, Codex also helped reconcile both reviews, clarify evaluation wording, and check formatting. No participant research results are claimed. Codex also assisted with the October 2 revisions addressing peer-review issues 11 and 12 and Aaron's browser/account scope decision. Review of these revisions by another team member is pending.

Kevin Kim used Codex for grammar, punctuation, and wording revisions. His exact model and reasoning settings were not specified in his contribution.

Verification and responsibility. The assistant checked cited sources, course requirements, document consistency, and PDF rendering. These checks do not establish that the proposed system works. Lucas and Kevin reviewed the Development Plan; their changes and reflections are incorporated here. Charter ratification remains to be confirmed. Before submission, the team will verify the final content and update this disclosure to reflect subsequent revisions and any additional AI use. The team remains responsible for the submitted work.

Appendix B: Reflection

The following reflections were contributed by Lucas during the team review.

1. Why is it important to create a development plan before starting?
   A strong development plan is important for many reasons. First, it is used to get all team members on the same page about the project's descriptions, goals, and scope, and it helps the team clarify administrative details such as team roles, responsibilities and communication expectations. Explicitly laying out these expectations at the start of the project helps prevent assumptions from causing misunderstandings later on. Additionally, it can be used by team members to see what fields or technologies they should brush up on before the project begins. Lastly, this project may seem overwhelming at first, but a development plan can help avoid analysis paralysis by encouraging the team to make decisions and begin concrete work.

2. What are the advantages and disadvantages of continuous integration and deployment (CI/CD) in your opinion?
   CI/CD offer many disadvantages and advantages. Some advantages include faster releases, an easier time merging work, and automation of repetitive tasks such as testing, builds, and deployment. Additionally, CI/CD encourages smaller changes that are easier to understand and review. However, there are also disadvantages; the initial setup can take some time, and there is a learning curve to setting up an effective CI/CD pipeline. These pipelines can create a false sense of security, as developers may assume that if the pipeline passes, the code is correct. However, if test coverage is incomplete, automated checks can miss bugs making it easier for faulty code to slip through.

3. What disagreements occurred in this deliverable and how were they resolved?
   Overall, there were no major disagreements in this deliverable. Some differences while brainstorming ideas were resolved through discussion and compromise. There was a minor concern over the uneven distribution of work for this deliverable. For future deliverables, the team will divide work more clearly beforehand and follow the expectations laid out in this document.

Appendix C: Team Charter

External goals

Produce a useful open-source contribution, a credible working prototype informed by blind users, strong engineering evidence, and a strong course outcome.

Attendance

Expectations. Attend weekly syncs prepared and on time. Give advance notice of lateness, early departure, or absence when possible. A written update and decision input can support flexible attendance; absence alone is not a failure to contribute.

Acceptable excuses. Illness, emergencies, accessibility needs, and communicated academic/work conflicts merit rescheduling or reassignment. Repeated unexplained silence or uncommunicated missed commitments call for a recovery conversation, rather than an assumption about personal motivation.

Emergency process. Notify the group when practical, identify work at risk if able, and let the group redistribute urgent tasks. Personal details need not be made public.

Accountability and teamwork

Quality. Weekly commitments specify outputs and acceptance criteria. Meeting preparation includes reading the agenda and relevant drafts and completing any additional duties associated with the member's meeting role for the week. Contributions include evidence of checks, understandable documentation, and a reviewable change. Drafts are ready 48 hours before deadlines. All members must understand and be able to explain AI-assisted work they submit.

Attitude. Discuss ideas respectfully, make room for disagreement, and ask before assuming a teammate's intent. Treat participant expertise as evidence; avoid speaking for visually impaired people without consulting them. Do not disclose personal participant information to resolve team debates.

Staying on track. Use weekly output commitments and the issue board, not fixed hours or commit quotas. Two unexplained missed commitments in one month trigger a private recovery conversation with an explicit revised plan. If the problem remains unresolved, involve the TA with factual records and an opportunity for the member to respond. Recognize useful contributions and help remove blockers.

Team building. Include short demos and informal check-ins in the weekly sync. Recognize progress; any additional social activities are optional and have not yet been agreed upon.

Decision making. Seek evidence and run a small experiment where useful, then vote if discussion does not resolve the question. For a tied technical choice, the assigned owner decides and records the rationale. Course-policy questions go to the TA. No individual can invent another member's consent or participant research results.